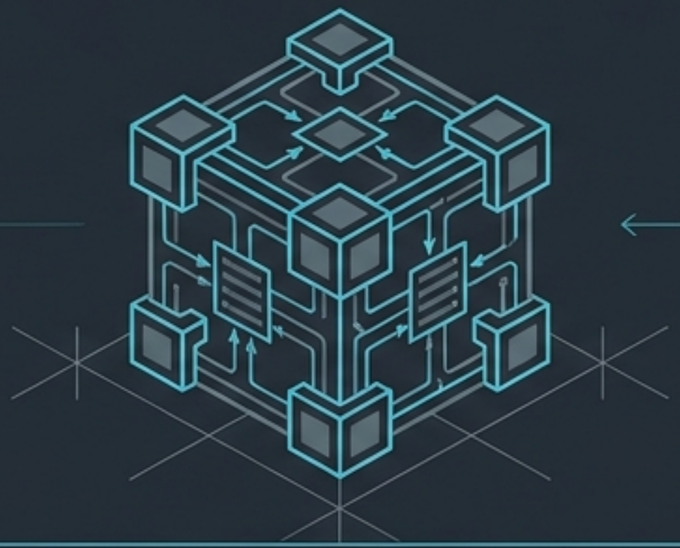
The SYSCRAFTERS Blueprint for SALESTORM

Primary Focus: High-Scale E-Commerce Flash Sale Architecture

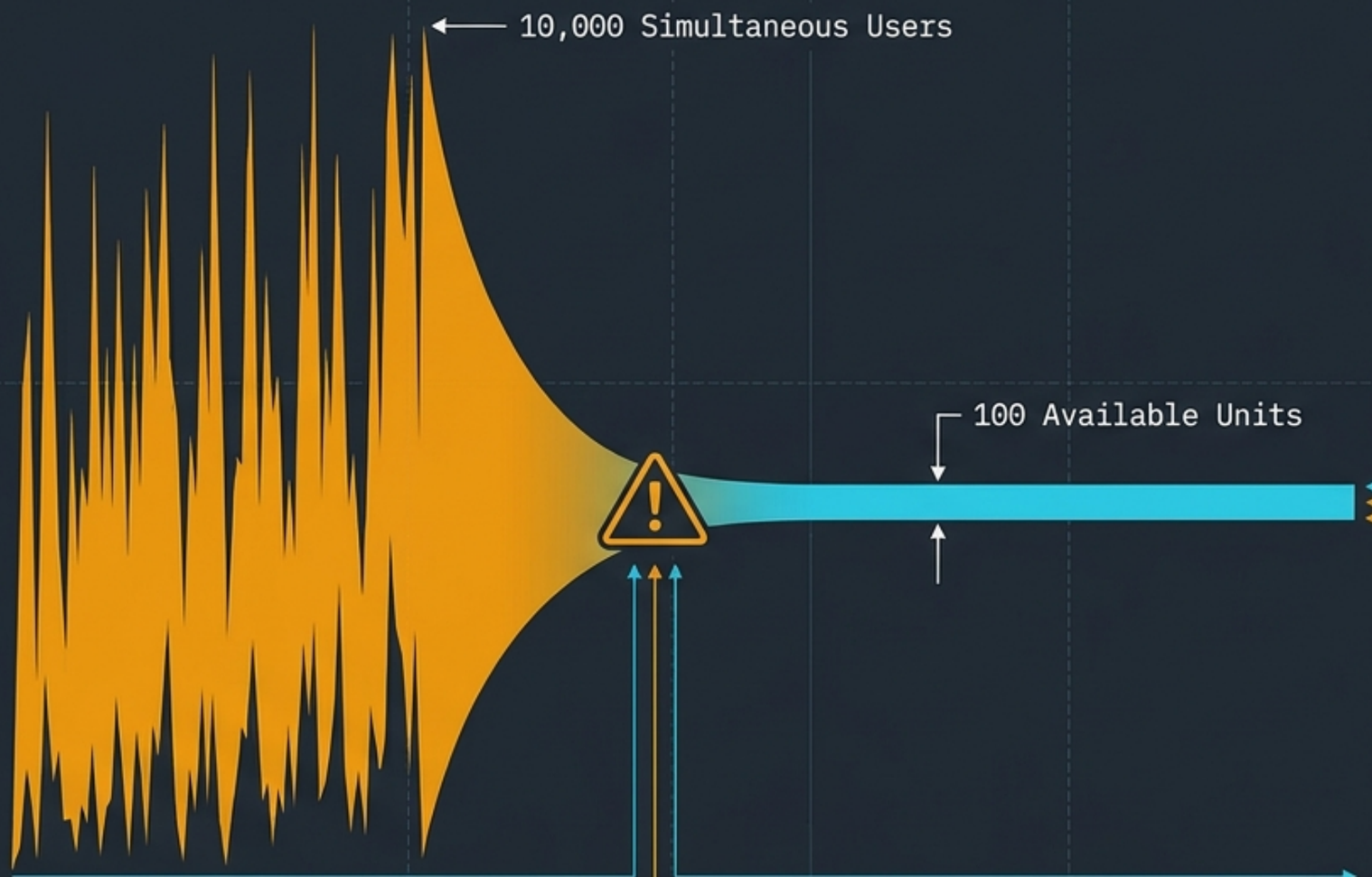


Engineering Ideology		
Design-First	AI-Assisted	Defensible Engineering
Architectural decisions precede implementation.	Strategic acceleration for prototypes and simulations, zero reliance for core logic.	Every trade-off is documented, tested, and justified.

Gemini Notebook

The Flash Sale Constraint: 10,000 Concurrent Requests, 100 Units

Telemetry Dashboard



The Stress Scenario

95%
Payment Success Rate

5%
Payment Failure Rate

2%
Duplicate Requests

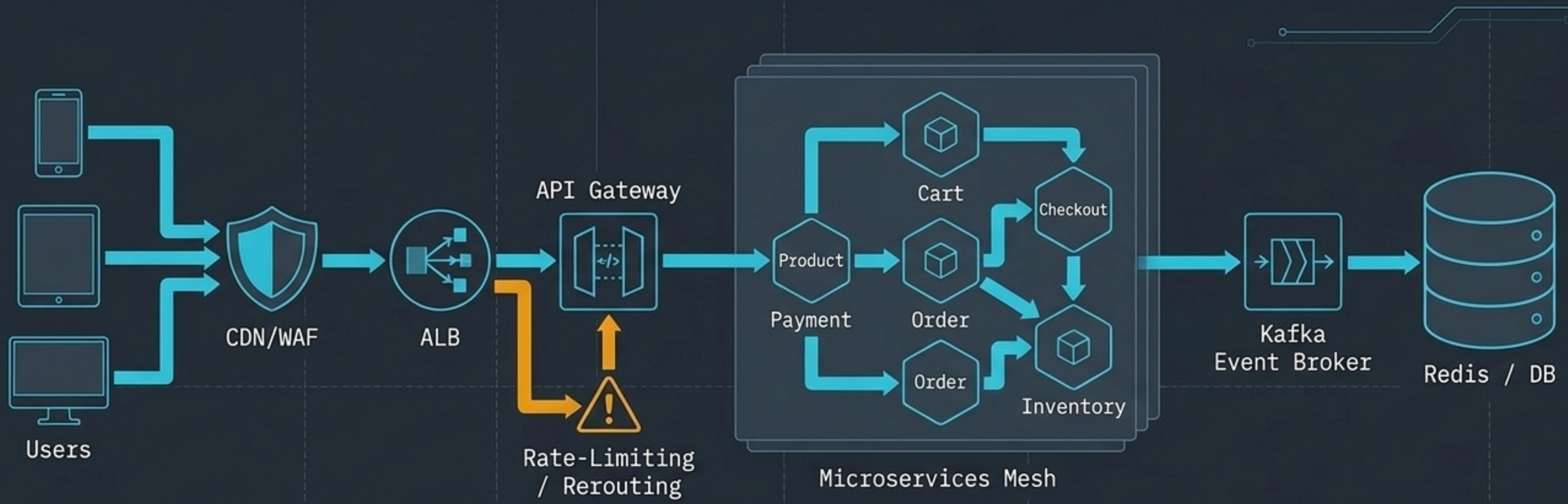
30s
Ozder Service Offline

The Absolute Directives

- [x] Zero overselling
- [x] Zero duplicate transactions
- [x] Zero dropped states under service failure

Gemini Notebook

Edge-to-Database Macro Architecture



Edge & Throttling

CDN caching and WAF protect the perimeter. The API Gateway enforces strict rate limiting to protect backend services.

Service Mesh

Decoupled SOA microservices handling localized domains (Product, Cart, Checkout, Payment, Order, Inventory).

Persistence & Messaging

Relational database for transactional integrity, Redis for fast locks, and Kafka as the asynchronous event broker for cross-service resilience.

Gemini Notebook

Concurrency Decision Matrix: Protecting Inventory

Requirement Context: The goal is “at most 100 successful sales,” NOT “strict first-come, first-serve.”

Approach A: Atomic DB Updates (Selected)

Mechanism: Database handles contention via row-level locks/atomic decrements.

Latency: Minimal (direct DB execution).

Complexity: Low. Guarantees no overselling natively.

Approach B: Queuing with Sequence Numbers (Rejected)

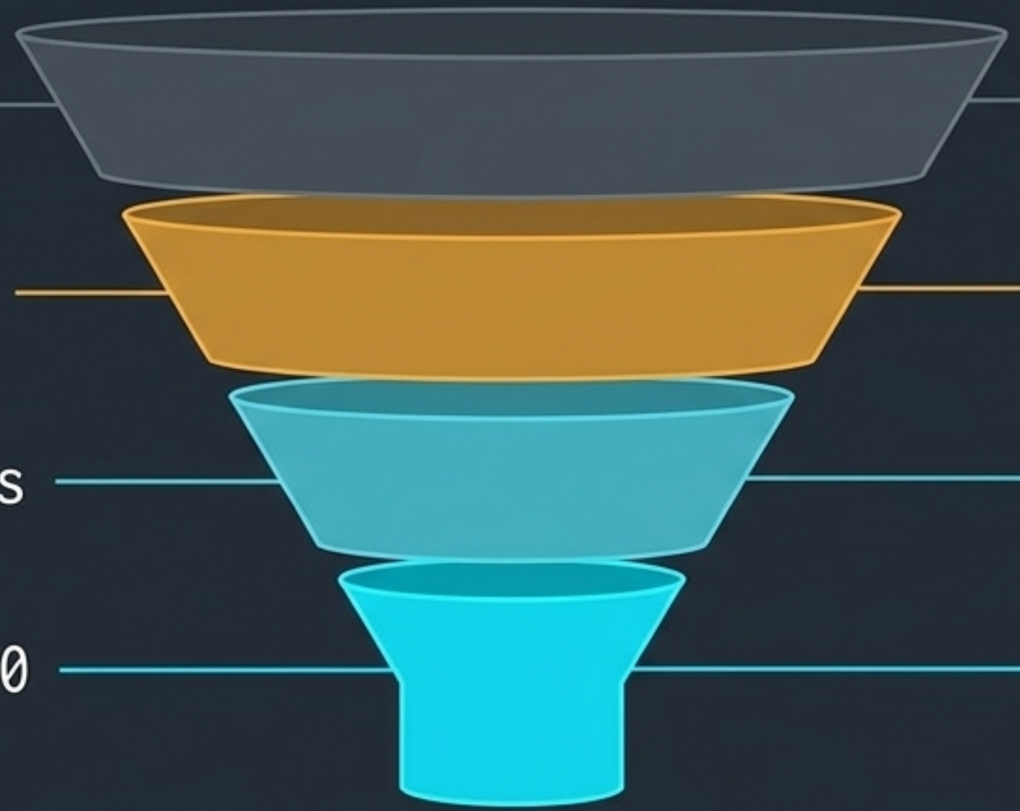
Mechanism: Serializes all requests into a queue to enforce strict arrival order.

Latency: Higher (adds queue processing overhead).

Complexity: High. Unnecessary unless strict FIFO fairness is a hard business requirement.

Gemini Notebook

The Zero Overselling Load Funnel



Level 1: Arrive at CDN/WAF. Edge caching absorbs product discovery traffic.

Level 2: Throttling sheds excess load to prevent backend server collapse.

Level 3: Validated requests hit the Inventory Service.

Level 4: Atomic Lock. Database sequentially applies update. Exactly 100 temporary reservations granted.

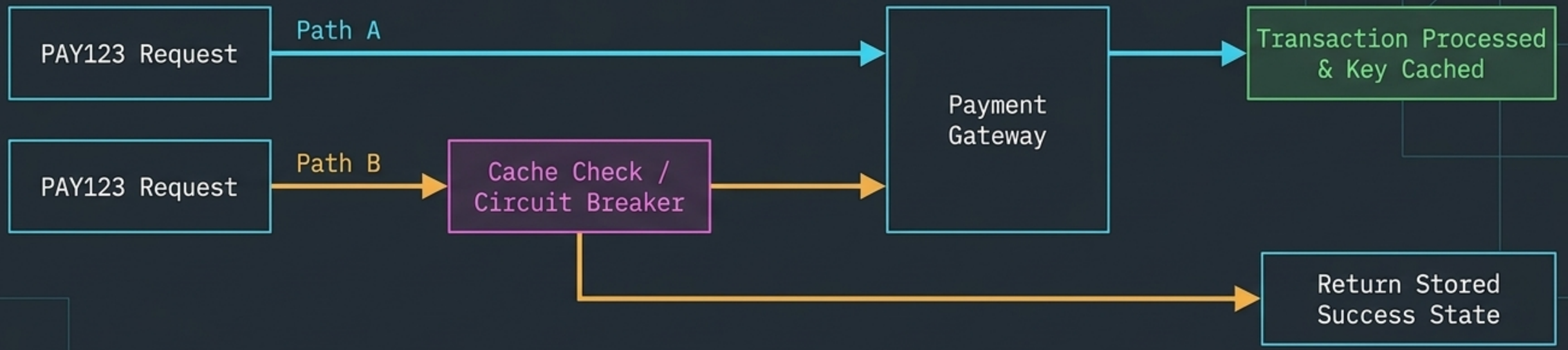


Auto-Release Protocol

Unpaid reservations automatically expire and release back to the pool via background workers, preventing gridlock.

Gemini Notebook

Idempotent Payments: Guaranteeing Financial Safety



The Threat

Network drops cause users to click 'Pay' twice.

Without idempotency, ₹500 + ₹500 = ₹1000 charged.

The Mechanism

The backend generates a unique idempotency_key (e.g., PURCHASE_ABC123) for each checkout attempt.

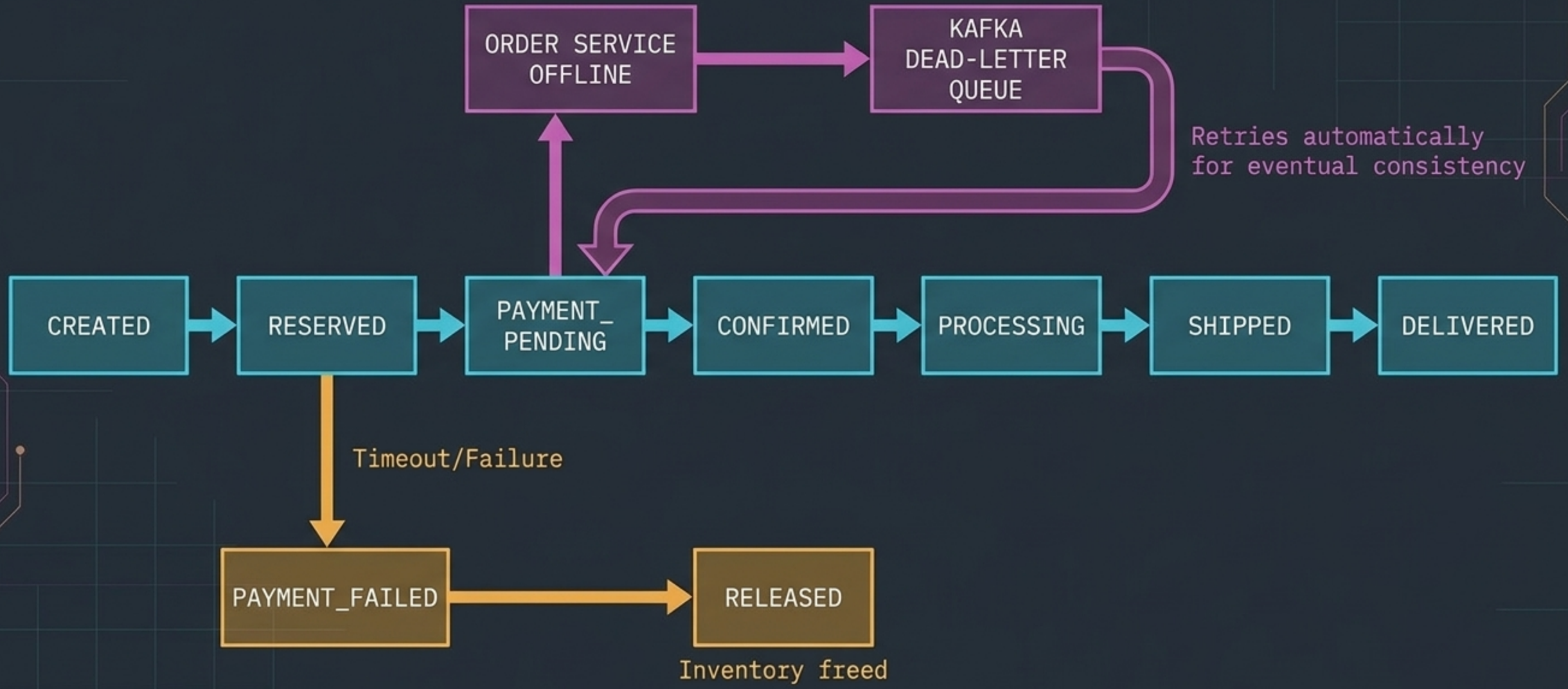
The Guarantee

The gateway stores this key.

Identical repeated requests safely return the cached previous result without double-charging.

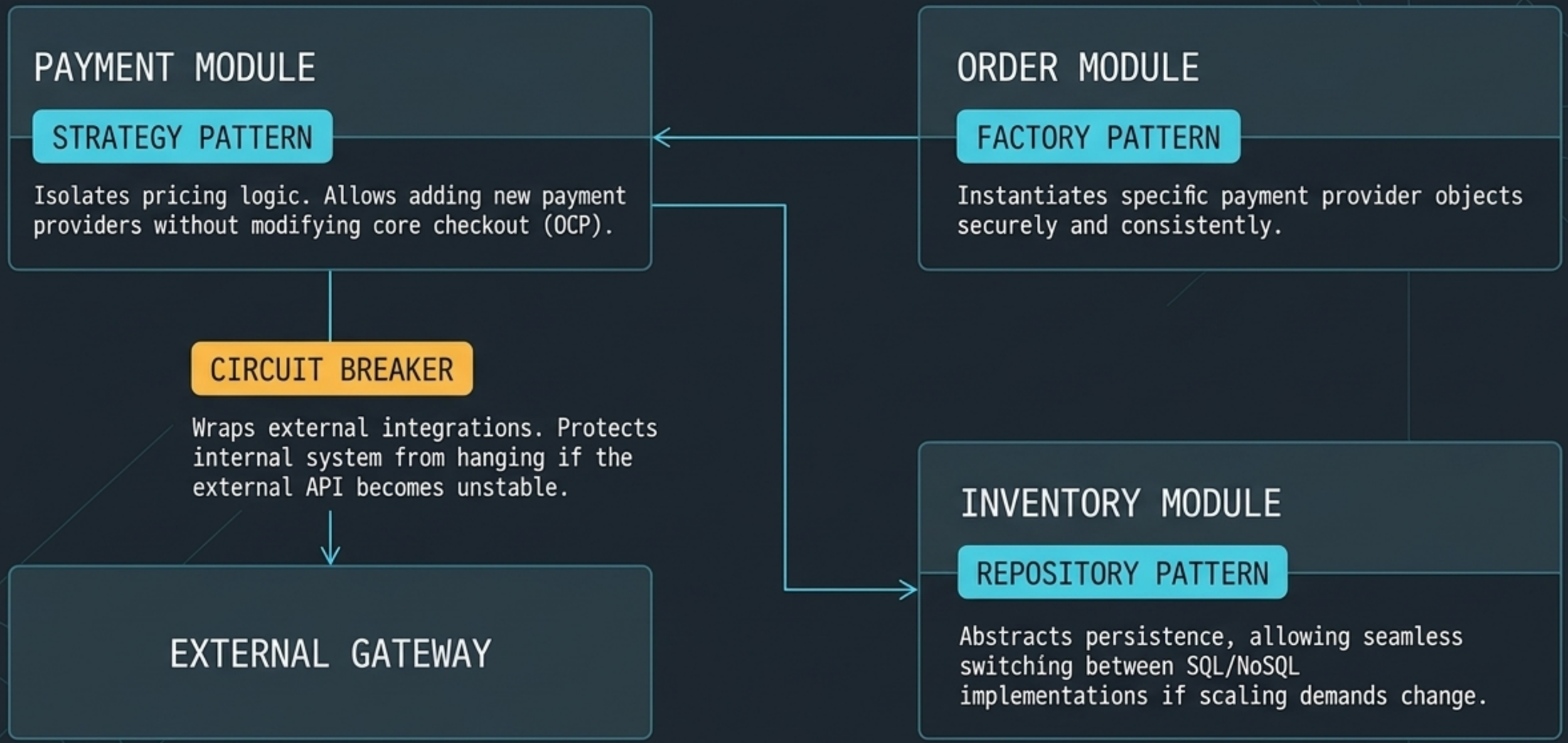
Gemini Notebook

Deterministic Order Lifecycle State Machine



Gemini Notebook

Structural Integrity via Low-Level Design Patterns



Gemini Notebook

Consistency > Availability

The Trade-off:

We traded absolute availability for strict consistency.

The Defense:

In a flash sale, overselling inventory damages brand trust and incurs manual refund overhead. We accept slightly higher latency at the checkout bottleneck to guarantee exactly 100 units via atomic locks.

Asynchronous > Synchronous

The Trade-off:

We traded synchronous simplicity for asynchronous resilience.

The Defense:

Synchronous chains shatter under 500k req/sec spikes. By decoupling Payment and Order services via Kafka, a 30-second downstream outage simply queues the work, preventing data loss.

Gemini Notebook



Inventory Protected

Atomic concurrency guarantees zero overselling.



Revenue Secured

Idempotency keys prevent double billing and financial anomalies.



Resilience Guaranteed

Event-driven architecture ensures zero data loss during localized service outages.

Architecture optimized for scale. Engineered for survival.

Gemini Notebook